Andrew Kozlevcar

11 - 11:50

In [1]:
```
import numpy as np
import pandas as pd
import plotly.express as px

from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, log_loss, confusion_matrix)

px.defaults.template = "plotly_white"

url = ("https://raw.githubusercontent.com/Bhavya-Chellapandian/"
       "online-shoppers-purchasing-intention-dataset/main/"
       "online_shoppers_intention.csv")

shopping = pd.read_csv(url)
shopping.head()
```

Out[1]:

	Administrative	Administrative_Duration	Informational	Informational_Duration	Proc
0	0	0.0	0	0.0	
1	0	0.0	0	0.0	
2	0	0.0	0	0.0	
3	0	0.0	0	0.0	
4	0	0.0	0	0.0	

Question 1

In [2]:
```
print("Dataset size:", shopping.shape)
print("\nPurchases and non-purchases:")
print(shopping["Revenue"].value_counts())
print("\nClass percentages:")
print((100 * shopping["Revenue"].value_counts(normalize=True)).round(1))

# Revenue is True for purchase and False otherwise.
y = shopping["Revenue"].astype(int)
X = shopping[["ProductRelated", "ProductRelated_Duration",
              "BounceRates", "ExitRates", "PageValues", "VisitorType"]]
```

```
cats = ["VisitorType"]
nums = ["ProductRelated", "ProductRelated_Duration",
        "BounceRates", "ExitRates", "PageValues"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=6021, stratify=y
)

always_no_purchase = np.zeros(len(y_test), dtype=int)
print("\nTest purchase rate:", round(y_test.mean(), 3))
print("Always-no-purchase test accuracy:",
      round(accuracy_score(y_test, always_no_purchase), 3))
print("Always-no-purchase test recall:",
      round(recall_score(y_test, always_no_purchase), 3))
```

```
Dataset size: (12330, 18)

Purchases and non-purchases:
Revenue
False    10422
True      1908
Name: count, dtype: int64

Class percentages:
Revenue
False    84.5
True     15.5
Name: proportion, dtype: float64

Test purchase rate: 0.155
Always-no-purchase test accuracy: 0.845
Always-no-purchase test recall: 0.0
```

Part A

15.5% of sessions ended in a purchase, whereas 84.5% of the sessions ended in no purchase. Class balance is particularly relevant to evaluation because many standard metrics, such as accuracy, do not control for the base rate, meaning that prediction performance on the larger group can obscure the performance on the smaller group. Thus, in cases of significant skew, it is important to leverage metrics that provide insight on performance in these sub-groups. In this example, we have a relatively small positive group, so we should use metrics like precision, recall, or F1 score to isolate performance on this outcome demographic.

Part B

The test accuracy for null model is 84.5%. However, its test recall is 0%. Together, these two metrics demonstrate the danger of accuracy as an isolated performance metric in the event of minority positive groups. At its core, accuracy has the sum of true positives

and true negatives in its numerator, which means it collapses the performance in these two groups; from accuracy alone, we cannot recover how well the model identified positives or negatives individually. Here, we have an extreme example in which our model always predicts False, which means that every negative is correctly classified and every positive is incorrectly classified. Since the negatives make up 84.5% of the test cohort, this results in a relatively high accuracy, and the disproprionate group share of negatives obscures the fact that none of the positives are correctly identified. Since recall takes true positives in the numerator, this results in the worst possible recall value of 0.

Part C

Since the company is concerned with identifying potential buyers, a model that predicts purchases better (possibly at the cost of identifying non-purchases) would be considered an improvement. Of course, there is a delicate balance; the company also does not want a model that just predicts purchase for each session, so the classifier needs to maintain some accuracy on the non-purchase group. Based on this goal, one metric that would be important to leverage is recall, which would denote the proportion of all true buyer sessions that were correctly identified. Similarly, it may be important to know the proportion of predicted purchases who were true purchases (precision) because the store might leverage resources on users labeled as potential purchases, and poor precision would lead to inefficient misallocation. Finally, a metric like specificity, which is the proportion of non-purchase sessions correctly identified, may be good to keep in mind to make sure that performance on non-purchases does not completely plummet.

Question 2

In [3]:
```
preprocess = ColumnTransformer(transformers=[
    ("encoder", OneHotEncoder(drop="first", handle_unknown="ignore"), cats),
    ("numeric", StandardScaler(), nums)
])

logreg = LogisticRegression(max_iter=5000, random_state=123)
pipe = Pipeline(steps=[("preprocess", preprocess), ("model", logreg)])
pipe.fit(X_train, y_train)

p_train = pipe.predict_proba(X_train)[:, 1]
p_test = pipe.predict_proba(X_test)[:, 1]
log_odds_test = pipe.decision_function(X_test)

threshold = 0.50
results_test = pd.DataFrame({
    "Actual_purchase": y_test.to_numpy(),
    "Probability_purchase": p_test,
```

```
    "Log_odds": log_odds_test,
    "Predicted_purchase": (p_test >= threshold).astype(int)
})
print("Predictions for the first eight test sessions:")
print(results_test.head(8).round(3).to_string(index=False))

comparison = pd.DataFrame({
    "Set": ["Train", "Test"],
    "Log loss": [log_loss(y_train, p_train), log_loss(y_test, p_test)],
    "Accuracy": [accuracy_score(y_train, p_train >= threshold),
                 accuracy_score(y_test, p_test >= threshold)]
})
print("\nTraining vs test metrics:")
print(comparison.round(3).to_string(index=False))
```

```
Predictions for the first eight test sessions:
 Actual_purchase  Probability_purchase  Log_odds  Predicted_purchase
               0                 0.131    -1.896                   0
               0                 0.037    -3.262                   0
               0                 0.108    -2.112                   0
               1                 0.138    -1.836                   0
               0                 0.070    -2.592                   0
               0                 0.136    -1.849                   0
               0                 0.143    -1.792                   0
               0                 0.005    -5.239                   0

Training vs test metrics:
  Set  Log loss  Accuracy
Train     0.304     0.884
 Test     0.290     0.887
```

Part A

For the first test session, its predicted probability is 0.131 and its predicted class is 0. In context, the model predicts that there is a 13.1% chance that this first test session will conclude in a purchase. Since the predicted probability does not match or exceed the classification threshold of 0.50, the first test session is predicted to be a non-purchase.

Part B

Positive predicted log odds imply that the predicted probability exceeds 0.5. If a session has positive log odds, then the model estimates that a session is more likely to end in a purchase than it is not. Alternatively, negative predicted log odds imply that the predicted probability is less than 0.5. If a session has negative log odds, then the model estimates that a session is less likely to end in a purchase than it is not.

Part C

The training log loss is 0.304, whereas the test log loss is 0.290; the test log loss is lower. A smaller log loss means that the model generally assigns higher probability to sessions that actually ended in purchase, assign lower probabilities to sessions that actually ended without a purchase, and avoids highly confident but incorrect predictions (sessions ending in purchase with predicted probability close to 0 or sessions not ending in purchase with predicted probability close to 1); it should be noted that log loss is an aggregated metric, which means that these statements may not be true for observations in a pointwise sense. Since the coefficients of the model are chosen to minimize the log loss on the training data, training log loss is not reliable indicator of out-of-sample performance; it is overly optimistic. To simulate a model's out-of-sample performance, one must evaluate its performance on unseen data, which gauges whether the model has captured population-level signal as opposed to just sample noise.

Question 3

In [4]:
```
y_pred_50 = (p_test >= 0.50).astype(int)

# Rows: actual purchase / no purchase. Columns: predicted purchase / no purc
cm = confusion_matrix(y_test, y_pred_50, labels=[1, 0])
cm_df = pd.DataFrame(cm,
    index=["Purchase", "No purchase"],
    columns=["Purchase", "No purchase"])
print("Confusion matrix at threshold 0.50:")
print(cm_df)

fig = px.imshow(cm_df, text_auto=True,
                color_continuous_scale="Blues",
                title="Test-set confusion matrix (threshold = 0.50)",
                labels=dict(x="Predicted", y="Actual", color="Count"))
fig.show()

print("\nAccuracy:", round(accuracy_score(y_test, y_pred_50), 3))
print("Precision:", round(precision_score(y_test, y_pred_50, zero_division=0
print("Recall:", round(recall_score(y_test, y_pred_50), 3))
print("F1 score:", round(f1_score(y_test, y_pred_50, zero_division=0), 3))
```

```
Confusion matrix at threshold 0.50:
             Purchase  No purchase
Purchase          141          241
No purchase        38         2046
Accuracy: 0.887
Precision: 0.788
Recall: 0.369
F1 score: 0.503
```

Part A

There are 141 true positives, 38 false positives, 241 false negatives, and 2046 true negatives. For this store, a false negative means that the model predicted that a session did not end in a purchase (in this case, predicted a probability of ending in purchase less than the threshold of 0.5) even though the session did actually end in a purchase.

Part B

The test precision was 0.788 whereas the test recall was 0.369. In plain language, of test sessions which the model (based on the set threshold) predicted ending in a purchase, 78.7% did actually end in a purchase. On the other hand, of test sessions which actually ended in a purchase, the model (based on the set threshold) predicted that 36.9% of them ended in a purchase. While the test accuracy is 88.7%, I previously noted that this metric collapses information about performance on the positive and negative groups. If one group is the majority, then the overall accuracy metric of a model will be mostly dictated by its performance on this majority. Here, we see that performance on the no purchase sessions, which make up the majority of the cohort (about 84.5%), is exceptionally strong (as signified by a true negative rate of about 98.2%), leading to an accuracy reading that somewhat obscures positive group performance.

Alternatively, precision and recall allow the business to better isolate performance on the actual purchase sessions, regardless of its size, in two important ways. The precision helps the business know how many of the predicted sessions ending in purchase actually end in purchase. As previously noted, the business may employ strategies to allocate resources to encourage purchase in sessions flagged as potential purchases, and so high precision ensures that resource allocation is selective and less wasteful. On the other hand, recall lets a business know the proportion of actual sessions ending in purchase that are correctly identified by the model. It is important for the business to know if the model has low recall in order to know the proportion of sessions ending in purchase the model is missing; in such a case, the model may not identify enough potential purchase sessions to make, for instance, additional resource allocation profitable.

Part C

The test F1 score is 0.503. The test F1 score is the harmonic mean of test precision and test recall. In this way, the metric collapses precision and recall into a single metric. Similar to accuracy, collapsing information may obscure the original metric scores. As discussed in part B, accuracy can be dominated by the majority group performance, but since F1 score is a metric composed of two (mostly) positive group performance metrics, it does a better job of isolating the performance on the positive group; note that precision still involves false postiives, which are actual negatives, so it does not entirely isolate positive group perfomance. Since F1 score is a harmonic mean, it can be shown

that low precision or low recall will pull the metric down, meaning the metric rewards models where they are balanced. This behavior of the metric is particularly useful as precision is often positively related with threshold and recall is often negatively related with threshold. Thus, as we span out thresholds, there is a precision-recall trade-off, and F1 score is a useful way to identify the balance point (assuming we equally value precision and recall).

Question 4

In [5]:
```
rows = []
for threshold in [0.25, 0.50]:
    predicted = (p_test >= threshold).astype(int)
    rows.append({
        "Threshold": threshold,
        "Predicted purchases": predicted.sum(),
        "Accuracy": accuracy_score(y_test, predicted),
        "Precision": precision_score(y_test, predicted, zero_division=0),
        "Recall": recall_score(y_test, predicted),
        "F1 score": f1_score(y_test, predicted, zero_division=0)
    })

threshold_table = pd.DataFrame(rows)
print(threshold_table.round(3).to_string(index=False))

predicted_25 = (p_test >= 0.25).astype(int)
cm_25 = confusion_matrix(y_test, predicted_25, labels=[1, 0])
cm_25_df = pd.DataFrame(cm_25,
    index=["Purchase", "No purchase"],
    columns=["Purchase", "No purchase"])
print("\nConfusion matrix at threshold 0.25:")
print(cm_25_df)

print("\nTest log loss at either threshold:",
      round(log_loss(y_test, p_test), 3))
```

```
 Threshold  Predicted purchases  Accuracy  Precision  Recall  F1 score
      0.25                  319     0.893      0.687   0.573     0.625
      0.50                  179     0.887      0.788   0.369     0.503

Confusion matrix at threshold 0.25:
             Purchase  No purchase
Purchase          219          163
No purchase       100         1984

Test log loss at either threshold: 0.29
```

Part A

At a threshold of 0.5, the number of predicted purchases is 179 sessions. At a threshold of 0.25, the number of predicted purchases is 319 sessions. At a threshold of 0.5, the

recall is 0.369. At a threshold of 0.25, the recall is 0.573. Thus, as threshold decreased from 0.5 to 0.25, the number of predicted purchases increased and the recall increased. Both of these trends make sense. Moreover, when we decrease our threshold, the sessions predicted to be purchases at threshold 0.5 will still be predicted pruchases at threshold 0.25; if a session was predicted to be a purchase at 0.5, this implies that its predicted probability of purchase was atleast 0.5 and thus is atleast 0.25. In this way, predicted purchases at 0.5 are a subset of the predicted purchases at 0.25. We may also gain additional predicted purchase sessions, namely those that the model gave a predicted probability between 0.25 and 0.5. Since recall counts the total proportion of actual purchase sessions that were predicted to be purchases (formally, $\frac{TP}{TP+FN} = \frac{TP}{AP}$, where AP is actual positives), its computation is an increasing function with respect to true positives. By the above argument, the number of true positives at 0.25 is atleast the number of true positives at 0.5, meaning recall should remain the same or increase.

Part B

At a threshold of 0.25, the precision is 0.687 and the F1 score is 0.625. At a threshold of 0.5, the precision is 0.788 and the F1 score is 0.503. Thus, precision is higher at a threshold of 0.5, but F1 score is higher at a threshold of 0.25. Missing a potential buyer corresponds to a false negative. Therefore, if we want to limit false negatives, we should choose a model with high recall, which measures the proportion of the actual sessions concluding in purchase that we accurately identified. In this case, the recall of threshold 0.25 is higher (by the previous question), so we should choose this treshold. Note that this improvement does not come without a trade-off. Namely, lowering the threshold effectively widens our "net", meaning, in the process of identifying more true positives, we also identify more false positives. This trade-off is reflected in the lower precision of the 0.25 threshold.

As a side note, I would not utilize F1 score because it effectively weights precision and recall equally, but the assignment explicitly says that we value recall more.

Part C

Empirically, the test log loss is 0.290 at both thresholds. More generally, changing the classification threshold does not change the test log loss. The test log loss of a given observation i is $-(y_i \log(p_i) + (1 - y_i) \log(1 - p_i))$. Thus, it is a function of p_i, where $p_i = \frac{1}{1+e^{-n_i}}$ and $n_i = \beta_0 + \sum_{j=1}^{p} \beta_j x_{ij}$. As a result, the log loss of a given test observation (and thus the average test log loss) is set once the coefficients are learned from the training data--an optimization process that only requires observed outcomes and features. All together, test log loss is a function of the test observations AND the training observations, since the latter yields the coefficients that produce the predicted probabilities for the given input features of the test observations.

Alternatively, the threshold is a condition imposed after the model is fit and indicates how to classify the outputted probabilites; with these classifications, we may then organize test observations into the four categories. Thus, in addition to the training observations and test observations (the former producing the learned coefficients and predicted probabilities for the latter's features), the confusion matrix is also a function of the classification threshold.